Algorithm Engineering
Woche 1, Aufgabe 1

Jan Vogt

Moore's Law (Folie 10)

Auf Folie 10 geht es um die Frage, warum man heute überhaupt parallel programmieren muss. Dort steht „The Free Lunch Is Over“ und darunter Moore's Law. In diesem Bericht gehe ich genauer auf Moore's Law ein und erkläre, wie es mit den vielen Kernen in heutigen Prozessoren zusammenhängt.

Was ist Moore's Law?

Gordon Moore, einer der Gründer von Intel, hat 1965 einen Artikel in der Zeitschrift Electronics veröffentlicht. Darin hat er beobachtet, dass sich die Anzahl der Transistoren auf einem Chip ungefähr jedes Jahr verdoppelt. Zehn Jahre später, also 1975, hat er seine Vorhersage auf eine Verdopplung alle zwei Jahre angepasst. In dieser Form kennt man Moore's Law auch heute noch.

Eigentlich ist es kein richtiges Gesetz wie in der Physik, sondern eher eine Beobachtung, die aber erstaunlich lange gestimmt hat. Der erste Mikroprozessor von Intel, der 4004 aus dem Jahr 1971, hatte gerade einmal etwa 2300 Transistoren, während große Chips heute mehrere zehn Milliarden davon haben.

Oft liest man auch, dass sich die Leistung von Prozessoren alle 18 Monate verdoppelt. Das hat Moore selbst so aber nie gesagt. Die 18 Monate stammen von einem Kollegen bei Intel, der zusätzlich mit eingerechnet hat, dass die Transistoren mit der Zeit auch schneller werden.

Warum werden Programme nicht mehr von alleine schneller?

Früher hat man von kleineren Transistoren automatisch profitiert, weil sie schneller schalten und gleichzeitig weniger Strom brauchen. Dieser Zusammenhang wird nach Robert Dennard (1974) als Dennard Scaling bezeichnet. Dadurch konnte man die Taktfrequenz immer weiter erhöhen, ohne dass der Chip zu heiß wurde. Programme sind also mit jeder neuen Prozessorgeneration schneller geworden, ohne dass man als Programmierer etwas dafür tun musste. Genau das ist mit dem „Free Lunch“ gemeint.

Um das Jahr 2005 hat das Dennard Scaling dann aufgehört zu funktionieren. Bei sehr kleinen Transistoren fließt nämlich auch dann Strom, wenn sie eigentlich ausgeschaltet sind (Leckströme), und die Chips wurden dadurch zu heiß. Deshalb ist die Taktfrequenz seitdem kaum noch gestiegen. Der Pentium 4 hatte 2004 schon 3,8 GHz und normale Prozessoren liegen heute nicht viel höher.

Moore's Law gilt aber weiterhin, es kommen also immer noch mehr Transistoren dazu. Die Frage war nur, was man mit den ganzen zusätzlichen Transistoren anfangen soll, wenn man den Takt nicht mehr erhöhen kann.

Die Formel auf der Folie

$$\text{power} = \text{performance}^{1.74}$$

Die Zahl kommt aus Messungen, die Intel an den eigenen Prozessoren gemacht hat (Grochowski und Annavaram). An der Formel sieht man ganz gut, warum es sich nicht lohnt, einen einzelnen Kern immer schneller zu machen. Soll ein Kern doppelt so schnell sein, braucht er $2^{1.74} \approx 3{,}3$ mal so viel Strom. Nimmt man stattdessen zwei Kerne mit der alten Geschwindigkeit, verdoppelt sich der Stromverbrauch nur ungefähr und man hat trotzdem die doppelte Leistung. Das funktioniert allerdings nur, wenn sich das Programm auch parallelisieren lässt.

Aus diesem Grund bauen die Hersteller seit etwa 2005 Prozessoren mit mehreren Kernen. Der Rechner im PC-Pool hat zum Beispiel 20 Kerne (8 P-Cores und 12 E-Cores).

Herb Sutter hat dazu 2005 den Artikel „The Free Lunch Is Over“ geschrieben. Seine Aussage ist, dass Programme nicht mehr von alleine schneller werden und man sie selbst so schreiben muss, dass sie mehrere Kerne nutzen, wenn man mehr Leistung haben will. Genau dafür lernen wir in der Vorlesung OpenMP.

Gilt Moore's Law heute noch?

Darüber wird ziemlich viel diskutiert. Die Verdopplung dauert inzwischen eher länger als zwei Jahre und neue Fertigungsprozesse werden immer teurer. Außerdem sind Namen wie „3 nm“ eher Marketing und haben mit einer echten Größe auf dem Chip nicht mehr viel zu tun. Der Chef von Nvidia hat 2022 zum Beispiel gesagt, dass Moore's Law tot ist, während Intel das anders sieht.

Fazit

Wichtig ist, dass Moore's Law eigentlich nur etwas über die Anzahl der Transistoren aussagt und nicht über die Geschwindigkeit. Mehr Transistoren gibt es zwar weiterhin, aber heute bekommt man sie vor allem in Form von mehr Kernen. Damit ein Programm davon auch etwas hat, muss man es selbst parallelisieren.